

MILESTONE 1

A1.The given schema

PatientRecords (

patient_id,

patient_name,

patient_phone,

doctor_name,

doctor_specialization,

appointment_date,

diagnosis,

treatment,

prescription_details,

medicine_name,

medicine_price,

bill_amount,

payment_status

)

A1: Identified issues are ,

- **Not 1NF not 2NF not 3NF**
- **Many data are there in one table**
- **No separate table for different types of data**
- **If we delete the data of doctor then all related data will vanish**
- **Updation and deletion is difficult**
- **No proper separation of entities**
- **If we wanna update doctors specialization we must update many rows**
- **Any changes update everywhere**
- **Cannot insert new doctor unless patient record exists**

We can classify the issue as:

1.Normalization issue

2.Data can be duplicated

3.Data cannot be updated easily

4.No separation of data

5.cannot delete efficiently

6.cannot insert independent entities

A2. Normalize the Schema

The given schema converting into 1NF

Duplicated data are doctor_name,doctor_specialization,diagnosis,treatment,medicin name

THE 1NF SCHEMA ARE:

- Patient(patient_id (pk),patient_name)
- Doctor(doctor_id,(pk),doctor_name)
- patientdoctor(patient_id(pk),doctor_id(pk));
- doctorspecialization(specialization_id(pk), specialization_name, doctor_id)
- phoneno(phone_id(pk),patient_id(fk),phone_no)
- appointment(appointment_id (PK), patient_id (FK), doctor_id (FK), appointment_date)
- diagnosis(diagnosis_id(pk), patient_id(fk),diagnosis_name)
- treatment(treatment_id(pk),treatment_name,diagnosis_id(fk),patient_id)
- medicinname(medicin_id(pk),prescription_id(fk),medicin_name,medicin_price)
- bill(bill_id(pk),patient_id(fk),bill_amount,payment_status)
- prescription(prscription_id(pk),diagonosis_id(fk),prescription_details);

THE 2NF SCHEMA ARE:

- Patient(patient_id(pk),patient_name)
- Phonenummer(phone_id(pk),patient_id(fk),phone_no)
- Doctor(doctor_id(pk),doctor_name,doctor_experience)
- Specialization(specialization_id(pk),specialization_name)
- Doctorspecialization(doctor_id(fk),specialization_id(fk))
- Primary key(doctor_id,specialization_id)
- Appointment(appointment_id(pk),patient_id(fk),doctor_id(fk),appointment_description)
- Diagonisis(diagnosis_id (pk),diagonisis_name)
- Patienddiagnosis(patient_id(fk),diagnosis_id(fk))
- Primarykey(patient_id,diagnosis_id)
- Treatment(treatment_id PK, treatment_name, diagnosis_id FK)
- Prescription(prescription_id PK, diagnosis_id FK, prescription_details)
- Medicine(medicine_id PK, medicine_name, medicine_price)
- Prescriptionmedicin(prescription_id(fk),medicine_id(fk))
- Primary key(prescription_id,medicine_id)
- Bill(bill_id(pk),patient_id(fk),bill_amount,payment_status)

The 3NF SCHEMA ARE:

- Patient(patient_id(pk),patient_name)
- Phonenumner(phone_id(pk),patient_id(fk),phone_no)
- Doctor(doctor_id(pk),doctor_name,doctor_experience)
- Specialization(specialization_id(pk),specialization_name)
- Doctorspecialization(doctor_id(fk), specialization_id(fk), PRIMARY KEY(doctor_id, specialization_id))
- Appointment(appointment_id(pk),patient_id(fk),doctor_id(fk),appointment_date,appointment_description)
- Diagonisis(diagnosis_id(pk),diagonisis_name)
- Patienddiagnosis(patient_id(fk),diagnosis_id(fk) ,Primarykey(patient_id,diagnosis_id))
- Treatment(treatment_id PK, treatment_name)
- Diagnosistreatment(diagnosis_id(fk),treatment_id(fk),Primarykey(diagnosis_id,treatment_id))
- Prescription(prescription_id PK, diagnosis_id FK, prescription_details)
- Medicine(medicine_id PK, medicine_name, medicine_price)
- Prescriptionmedicin(prescription_id(fk),medicine_id(fk) ,primarykey(prescription_id,medicin_id))
- Bill(bill_id(pk),patient_id(fk),bill_amount,payment_status)

A3.Assumption

- 1.I have assumed that the doctors can have be specialized in multiple field
- 2.I have assumed that prescriptionid depends on diagnosisid
- 3.Patient can consult multiple doctors and doctor can treat multiple patients.
- 4.A prescription is given for a specific diagnosis.
- 5.Each bill is generated for specific patient which include totalbillamount and paymentstatus

SECTION B — DATABASE DESIGN FROM REQUIREMENTS

B1. Requirement Clarification

- Patient have multiple phone numbers
- Should I collect patients details like Addresss,age,gender ?
- Can patient visit multiple doctors?
- Can doctors have multiple specializations?
- Should doctors detailed should be included like experience qualification etc?
- can appointments be cancelled or rescheduled?

B2. Design

- Patient(patient_id PK, patient_name, age, gender, address)
- PhoneNumber(phone_id PK, patient_id FK, phone_no)
- Doctor(doctor_id PK, doctor_name, doctor_experience)

- Department(department_id PK, department_name)
- DoctorDepartment(doctor_id FK, department_id FK,
- PRIMARY KEY(doctor_id, department_id))
- Specialization(specialization_id PK, specialization_name)
- DoctorSpecialization(doctor_id FK, specialization_id FK,
- PRIMARY KEY(doctor_id, specialization_id))
- Appointment(appointment_id PK, patient_id FK, doctor_id FK, appointment_date, appointment_time, status)
- Diagnosis(diagnosis_id PK, diagnosis_name)
- Treatment(treatment_id PK, treatment_name)
- DiagnosisTreatment(diagnosis_id FK, treatment_id FK,
- PRIMARY KEY(diagnosis_id, treatment_id))
- Prescription(prescription_id PK, appointment_id FK, prescription_details)
- Medicine(medicine_id PK, medicine_name, medicine_price)
- PrescriptionMedicine(prescription_id FK, medicine_id FK, dosage, duration,
- PRIMARY KEY(prescription_id, medicine_id))
- Bill(bill_id PK, patient_id FK, total_amount, payment_status)

B3. Normalization

Update During Development

The hospital has introduced an operational change.

A single consultation can involve multiple doctors, and each doctor must be associated with their contribution to the consultation.

Update your design to accommodate this change.

So updated schema is

Separated appointment and consultation, introduced multi-doctor support via a junction table with contribution, and correctly linked diagnosis, prescription, and billing to consultation instead of appointment/patient.

ConsultationDoctor(consultation_id FK, doctor_id FK, contribution, PRIMARY KEY(consultation_id, doctor_id))

consultation(consultation_id PK, appointment_id FK, notes)

consultationdoctor(consultation_id FK, doctor_id FK, contribution)

consultationdiagnosis(consultation_id FK, diagnosis_id FK)

prescription(consultation_id FK)

bill(patient_id FK, consultation_id FK)

The sql queries for tables are

```
create table patient (  
    patient_id int primary key,  
    patient_name varchar(100),  
    age int,  
    gender varchar(10),  
    address varchar(255)  
);
```

```
create table phonenumber (  
    phone_id int primary key,  
    patient_id int,  
    phone_no varchar(15),  
    foreign key (patient_id) references patient(patient_id)  
);
```

```
create table doctor (  
    doctor_id int primary key,  
    doctor_name varchar(100),  
    doctor_experience int  
);
```

```
create table department (  
    department_id int primary key,  
    department_name varchar(100)  
);
```

```
create table doctordepartment (  
    doctor_id int,  
    department_id int,  
    primary key (doctor_id, department_id),
```

```
foreign key (doctor_id) references doctor(doctor_id),  
foreign key (department_id) references department(department_id)  
);
```

```
create table specialization (  
    specialization_id int primary key,  
    specialization_name varchar(100)  
);
```

```
create table doctorspecialization (  
    doctor_id int,  
    specialization_id int,  
    primary key (doctor_id, specialization_id),  
    foreign key (doctor_id) references doctor(doctor_id),  
    foreign key (specialization_id) references specialization(specialization_id)  
);
```

```
create table appointment (  
    appointment_id int primary key,  
    patient_id int,  
    doctor_id int,  
    appointment_date date,  
    appointment_time time,  
    status varchar(20),  
    foreign key (patient_id) references patient(patient_id),  
    foreign key (doctor_id) references doctor(doctor_id)  
);
```

```
create table consultation (  
    consultation_id int primary key,  
    appointment_id int,
```

```
notes text,  
foreign key (appointment_id) references appointment(appointment_id)  
);
```

```
create table diagnosis (  
    diagnosis_id int primary key,  
    diagnosis_name varchar(100)  
);
```

```
create table consultationdiagnosis (  
    consultation_id int,  
    diagnosis_id int,  
    primary key (consultation_id, diagnosis_id),  
    foreign key (consultation_id) references consultation(consultation_id),  
    foreign key (diagnosis_id) references diagnosis(diagnosis_id)  
);
```

```
create table treatment (  
    treatment_id int primary key,  
    treatment_name varchar(100)  
);
```

```
create table diagnosistreatment (  
    diagnosis_id int,  
    treatment_id int,  
    primary key (diagnosis_id, treatment_id),  
    foreign key (diagnosis_id) references diagnosis(diagnosis_id),  
    foreign key (treatment_id) references treatment(treatment_id)  
);
```

```
create table prescription (  
    
```

```
prescription_id int primary key,  
consultation_id int,  
prescription_details text,  
foreign key (consultation_id) references consultation(consultation_id)  
);
```

```
create table medicine (  
medicine_id int primary key,  
medicine_name varchar(100),  
medicine_price decimal(10,2)  
);
```

```
create table prescriptionmedicine (  
prescription_id int,  
medicine_id int,  
dosage varchar(50),  
duration varchar(50),  
primary key (prescription_id, medicine_id),  
foreign key (prescription_id) references prescription(prescription_id),  
foreign key (medicine_id) references medicine(medicine_id)  
);
```

```
create table bill (  
bill_id int primary key,  
patient_id int,  
consultation_id int,  
total_amount decimal(10,2),  
payment_status varchar(20),  
foreign key (patient_id) references patient(patient_id),  
foreign key (consultation_id) references consultation(consultation_id)  
);
```



```

create table consultationdoctor (

consultation_id int,

doctor_id int,

contribution varchar(100),

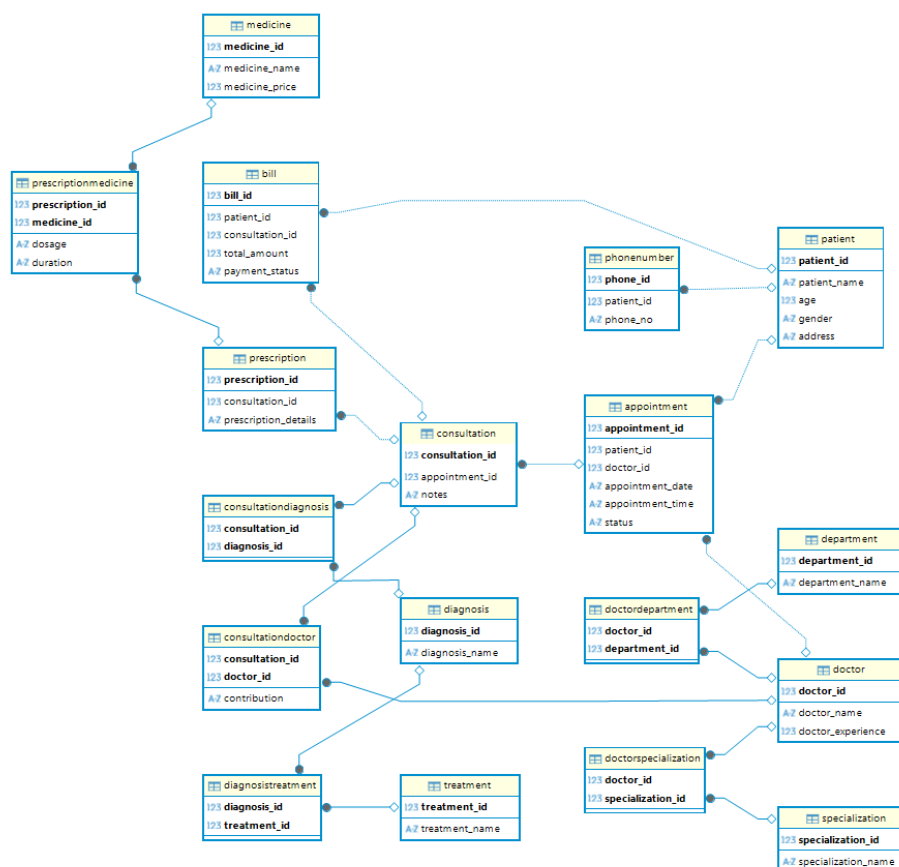
primary key (consultation_id, doctor_id),

foreign key (consultation_id) references consultation(consultation_id),

foreign key (doctor_id) references doctor(doctor_id)

);

```



SECTION C — QUERIES AND TRANSACTIONS

1) Retrieve the complete consultation history of a patient, including doctors consulted, diagnoses, and prescribed medicines.

(Must use joins)

```

select p.patient_name,

       c.consultation_id,

```

```

    d.doctor_name,
    dg.diagnosis_name,
    m.medicine_name
from patient p
join appointment a on p.patient_id = a.patient_id
join consultation c on a.appointment_id = c.appointment_id
join consultationdoctor cd on c.consultation_id = cd.consultation_id
join doctor d on cd.doctor_id = d.doctor_id
join consultationdiagnosis cdiag on c.consultation_id = cdiag.consultation_id
join diagnosis dg on cdiag.diagnosis_id = dg.diagnosis_id
join prescription pr on c.consultation_id = pr.consultation_id
join prescriptionmedicine pm on pr.prescription_id = pm.prescription_id
join medicine m on pm.medicine_id = m.medicine_id;

```

1 select p.patient_name, c.consultation_id, d.doctor_name, dg.diagnosis_name, m.medicine_name

	AZ patient_name	123 consultation_id	AZ doctor_name	AZ diagnosis_name	AZ medicine_name
1	rahul	1	dr ravi	fever	paracetamol
2	anita	2	dr meena	migraine	ibuprofen
3	kiran	3	dr arun	fracture	antibiotic
4	sneha	4	dr priya	cold	cough syrup
5	arjun	5	dr vinay	allergy	skin cream

2. Identify the doctor who has handled the highest number of consultations.
(Must use aggregation and joins)

```

select d.doctor_name,
    count(cd.consultation_id) as total_consultations
from doctor d
join consultationdoctor cd on d.doctor_id = cd.doctor_id
group by d.doctor_id, d.doctor_name
order by total_consultations desc;

```

	AZ doctor_name ▼	123 total_consultations ▼	
1	dr ravi	2	
2	dr meena	1	
3	dr arun	1	
4	dr priya	1	
5	dr vinay	1	

3. List all patients who have pending payments along with total outstanding amount.
(Must use joins and grouping)

```
select p.patient_name,
       sum(b.total_amount) as pending_amount
from patient p
join bill b on p.patient_id = b.patient_id
where b.payment_status = 'pending'
group by p.patient_id, p.patient_name;
```

	AZ patient_name ▼	123 pending_amount ▼	
1	rahul	1,500	
2	kiran	1,500	
3	arjun	600	

4. Write a transaction to handle a consultation and billing process:

- Insert consultation record
- Insert associated diagnoses
- Insert prescribed medicines
- Generate bill
- Record payment status
- Ensure the process is consistent in case of failure

begin;

```
insert into consultation (consultation_id, appointment_id, notes)
values (6, 1, 'general checkup');
```

```
insert into consultationdoctor (consultation_id, doctor_id, contribution)
values (6, 1, 'primary');
```

```
insert into consultationdiagnosis (consultation_id, diagnosis_id)
values (6, 1);
```

```
insert into prescription (prescription_id, consultation_id, prescription_details)
values (6, 6, 'take medicines regularly');
```

```
insert into prescriptionmedicine (prescription_id, medicine_id, dosage, duration)
values (6, 1, '2 times a day', '5 days');
```

```
insert into bill (bill_id, patient_id, consultation_id, total_amount, payment_status)
values (6, 1, 6, 500.00, 'pending');
```

```
commit;
```

```
rollback;
```